

1. Actividad de experimentación matemática

Registrar observaciones para las siguientes configuraciones:

Prueba	Acceleration	Deceleration	Pregunta
A	400	400	¿El personaje se siente pesado o ligero?
B	1400	1800	¿Existe respuesta rápida sin detenerse bruscamente?
C	4000	4000	¿Se aproxima a un movimiento instantáneo?
D	800	3000	¿Cómo cambia la sensación al arrancar y detenerse?

No existe un único conjunto de valores correcto. Los parámetros dependen del género, tamaño de los sprites, escala del mundo y sensación deseada.

Pregunta	Respuesta
¿El personaje se siente pesado o ligero?	El personaje presenta un movimiento ligeramente pesado, ya que la aceleración y la desaceleración son moderadas.
¿Existe respuesta rápida sin detenerse bruscamente?	El movimiento se percibe un poco brusco, debido a que el personaje acelera y se detiene con mayor rapidez.
¿Se aproxima a un movimiento instantáneo?	El personaje se mueve demasiado rápido, aproximándose a un movimiento casi instantáneo.
¿Cómo cambia la sensación al arrancar y detenerse?	El personaje presenta una sensación similar a un deslizamiento, debido a la diferencia entre la aceleración y la desaceleración.

2. Pruebas funcionales obligatorias

Prueba	Resultado esperado	Estado
Entrada	Las cuatro acciones responden correctamente	El personaje se desplaza correctamente en las cuatro direcciones.
Diagonal	No existe aumento artificial de rapidez	El personaje puede desplazarse correctamente en las cuatro direcciones diagonales.
Aceleración	El jugador alcanza max_speed progresivamente	La velocidad del personaje aumenta progresivamente mientras se mantiene presionada una dirección.
Desaceleración	El jugador se detiene progresivamente	Al soltar la dirección, el personaje reduce progresivamente su velocidad hasta detenerse.
Muros	No puede atravesar StaticBody2D	El personaje colisiona correctamente con las paredes y no las atraviesa.
Deslizamiento	Puede continuar junto a una pared al entrar en diagonal	Al desplazarse diagonalmente contra una pared, el personaje se desliza a lo largo de ella.
Capas	Player detecta World con la máscara configurada	Las capas y máscaras de colisión permiten una interacción correcta entre el Player y los muros.
Cámara	Sigue al jugador sin saltos bruscos	La Camera2D sigue correctamente al personaje durante su desplazamiento.
Límites	No revela zonas externas al nivel	Los límites impiden que la cámara muestre el exterior del escenario.

3. Preguntas de reflexión

1. **¿Por qué CharacterBody2D no se comporta igual que RigidBody2D?**

CharacterBody2D está diseñado para controlar directamente el movimiento de un personaje mediante código, permitiendo definir su velocidad, aceleración y dirección, mientras que RigidBody2D está pensado para comportarse de acuerdo con las leyes de la física, utilizando fuerzas, masa e impulsos. Por ello, CharacterBody2D ofrece un movimiento más controlado y predecible para un jugador.

2. **¿Qué diferencia existe entre direction y velocity?**

Direction representa la dirección hacia la que queremos que se mueva el personaje, mientras que velocity representa tanto la dirección como la rapidez del movimiento. Por ejemplo, una dirección puede indicar que el personaje debe desplazarse hacia la derecha, mientras que la velocidad determina qué tan rápido lo hará. En nuestro código, la dirección de entrada se multiplica por max_speed para obtener la velocidad objetivo.

3. **¿Por qué acceleration * delta produce un comportamiento más consistente?**

Porque delta representa el tiempo transcurrido entre frames y permite que la aceleración dependa del tiempo en lugar de depender directamente de la cantidad de frames por segundo. Al utilizar acceleration * delta, el personaje puede acelerar de manera más consistente, aunque el juego se ejecute con diferentes velocidades de FPS, evitando diferencias importantes en el movimiento.

4. **¿Por qué no debemos llamar move_and_slide(velocity * delta)?**

Porque en CharacterBody2D la función move_and_slide() utiliza directamente la propiedad velocity para realizar el movimiento y manejar las colisiones. Por eso primero establecemos o modificamos velocity y después llamamos a move_and_slide(). Multiplicar nuevamente velocity por delta no es necesario y puede producir un movimiento incorrecto.

5. **¿Qué diferencia conceptual existe entre Collision Layer y Collision Mask?**

Collision Layer indica en qué capa de colisión se encuentra un objeto, mientras que Collision Mask indica qué capas puede detectar o con cuáles puede interactuar. Por ejemplo, podemos colocar al jugador en una capa y configurar su máscara para detectar los muros en otra capa.

6. **¿Qué información aporta la normal de una colisión?**

La normal de una colisión indica la dirección perpendicular a la superficie donde ocurrió el contacto. Esta información permite saber desde qué orientación se produjo la colisión, por ejemplo, si el personaje chocó contra una pared, el suelo o el techo. En Godot se puede obtener mediante collision.get_normal() y utilizarla para analizar o modificar el comportamiento del personaje después de una colisión.

7. **¿Cuándo convendría separar Camera2D del nodo Player?**

Conviene separar Camera2D del nodo Player cuando el proyecto necesita un control más avanzado de la cámara, como seguimiento suave, zoom independiente, transiciones, screen shake o cambiar el objeto que la cámara sigue. Para un proyecto pequeño, colocar Camera2D como hija del Player es suficiente, pero separarla permite desarrollar comportamientos de cámara más flexibles.